

Banco de Dados NoSQL: ArangoDB

Conceitos, Classificação CAP, Casos de Uso, Ecossistema e Aplicação
CRUD Prática

Trabalho de Banco de Dados Não Relacionais

Junho de 2026

Orientador:

Tássio Sirqueira

Alunos:

Arthur José Barbosa Medeiros

Bernardo Vizeu de Miranda

Ícaro de Alencar Araripe Andrade

Pedro Henrique Cosele Ciampi

Sumário

1. Introdução
2. Tipo de Banco de Dados
3. Classificação Segundo o Teorema CAP
4. Casos de Uso e Indicações de Aplicação
5. Ferramentas de Gerenciamento e Ecossistema
6. Execução Prática com Docker
7. Aplicação CRUD em Python (python-arango)
8. Conclusão
9. Referências

1. Introdução

O ArangoDB é um sistema de gerenciamento de banco de dados NoSQL de código aberto, mantido pela empresa Arango, que se destaca por adotar uma arquitetura multi-modelo nativa: um único mecanismo de armazenamento e uma única linguagem de consulta declarativa, a AQL (ArangoDB Query Language), oferecem suporte simultâneo a documentos JSON, grafos e estruturas chave-valor.

Este documento apresenta os conceitos fundamentais do ArangoDB solicitados no trabalho: seu tipo de banco de dados, sua classificação segundo o teorema CAP, os casos de uso mais indicados, as ferramentas que compõem o seu ecossistema, a execução prática via Docker e, por fim, a arquitetura de uma aplicação CRUD completa desenvolvida em Python, que acompanha este documento em um projeto separado.

2. Tipo de Banco de Dados

Diferentemente de bancos NoSQL especializados em um único modelo, o ArangoDB foi projetado desde sua origem como um banco de dados multi-modelo nativo. Isso significa que os modelos de documento, grafo e chave-valor não são simulados por camadas externas, mas compartilham o mesmo motor de armazenamento e a mesma linguagem de consulta, permitindo combinar os três modelos em uma única operação.

2.1 Modelo de Documentos

Cada registro é armazenado como um documento no formato JSON, sem exigência de esquema fixo (schema-less), podendo conter campos do tipo array e subdocumentos hospedados em qualquer profundidade. Esse modelo é adequado para representar entidades de negócio com estrutura variável, como os exemplos de clientes e pedidos desenvolvidos na aplicação prática (seção 7).

2.2 Modelo de Grafos

Relacionamentos entre documentos são representados por meio de coleções de arestas (edge collections), que conectam vértices através dos atributos especiais `_from` e `_to`. Sobre essas coleções, o ArangoDB executa travessias nativas (graph traversals), cálculo de caminhos mais curtos e consultas de múltiplos saltos com desempenho superior ao de junções tradicionais baseadas apenas em comparação de chaves, especialmente em relações N->N profundas.

2.3 Modelo Chave-Valor

Documentos simples, identificados apenas por uma chave (`_key`), também podem ser tratados como pares chave-valor convencionais, sendo úteis para caches, contadores, sessões ou tabelas de configuração, sem a necessidade de um banco de dados adicional dedicado.

2.4 Síntese

Em síntese, o ArangoDB classifica-se como um banco de dados NoSQL multi-modelo, com suporte nativo e unificado a documentos, grafos e chave-valor, consultados por meio de uma única linguagem (AQL), reduzindo a necessidade de combinar múltiplos sistemas especializados em uma mesma arquitetura de aplicação.

3. Classificação Segundo o Teorema CAP

O teorema CAP estabelece que um sistema de dados distribuído não pode garantir simultaneamente, na presença de uma partição de rede, as três propriedades a seguir: Consistência (todas as réplicas refletem a escrita mais recente ou retornam erro), Disponibilidade (toda requisição recebe uma resposta, mesmo que não seja a versão mais recente) e Tolerância a Partição (o sistema continua operando mesmo quando nós não conseguem se comunicar entre si).

3.1 ArangoDB em Modo Single-Server

Em uma instância única (single-server), não há múltiplos nós nem, portanto, possibilidade de partição de rede interna, o teorema CAP, neste caso, não se aplica no sentido estrito em que foi formulado, já que ele descreve um trade-off específico de sistemas distribuídos. Esse modo oferece consistência total e alta disponibilidade local, mas não há tolerância a partição simplesmente porque não existe um cluster a ser particionado.

3.2 ArangoDB em Modo Cluster: Classificação CP

Em implantações em cluster, a documentação oficial do ArangoDB classifica a arquitetura como CP (Consistência + Tolerância a Partição), em um modelo master/master sem ponto único de falha. O cluster é composto por DB-Servers (armazenam os dados), Coordinators (recebem as requisições dos clientes e as distribuem) e Agents (executam o algoritmo de consenso Raft, responsável pela eleição de líder e pela supervisão do cluster).

A replicação entre réplicas de uma mesma coleção (shard) é síncrona: uma escrita só é confirmada ao cliente depois que o líder replica a alteração para todos os seguidores. Isso garante consistência forte e alta disponibilidade em operação normal, ao custo de uma latência de escrita maior. Na presença de uma partição de rede, o ArangoDB prioriza a consistência interna dos dados, podendo recusar uma operação em vez de devolver uma resposta potencialmente desatualizada, por isso sua classificação como CP, e não AP.

3.3 Configurabilidade do Trade-off

Embora a postura padrão do cluster seja CP, o ArangoDB permite ajustar parâmetros como o fator de replicação (replicationFactor) por coleção e o uso de satellite collections (coleções totalmente replicadas em todos os DB-Servers, úteis para dados de referência pouco alterados), possibilitando equilibrar consistência, desempenho e disponibilidade conforme a necessidade de cada coleção dentro da mesma aplicação.

3.4 Comparação com Outros Bancos NoSQL

Banco de Dados	Classificação predominante	Observação
ArangoDB (cluster)	CP	Replicação síncrona via consenso Raft; prioriza consistência em partições.
MongoDB (replica set)	CP (ajustável)	Write/read concern configuráveis permitem aproximar-se de AP.
Cassandra	AP (ajustável)	Consistência eventual por padrão; nível de consistência ajustável por consulta.
Couchbase	CP/AP (ajustável)	Comportamento varia conforme o nível de durabilidade configurado.
Neo4j (causal cluster)	CP	Consenso Raft entre núcleos; prioriza consistência causal.

4. Casos de Uso e Indicações de Aplicação

Por reunir documentos e grafos nativamente, o ArangoDB é especialmente indicado para domínios em que dados semiestruturados convivem com relacionamentos complexos e consultados com frequência:

- Redes sociais e motores de recomendação, em que conexões entre usuários, produtos e interesses são consultadas via travessias de grafo.
- Detecção de fraude, exigindo consultas de múltiplos saltos (ex.: encontrar cadeias de transações suspeitas) com baixa latência.
- Gestão de identidade e controle de acesso (IAM), modelando papéis, grupos e permissões como um grafo de relacionamentos.
- Catálogos de produtos e e-commerce, com documentos ricos em arrays e subdocumentos (variações, itens de pedido, avaliações).
- Logística e roteamento, aproveitando algoritmos nativos de caminho mais curto sobre grafos de rede.
- Bases de conhecimento, metadados de TI e CMDBs (Configuration Management Databases), que se beneficiam de relações ricas entre ativos.
- Aplicações de IoT e registro de eventos, combinando o modelo chave-valor para leituras rápidas com o modelo documental para metadados.

Por outro lado, o ArangoDB tende a ser menos indicado para cargas analíticas massivas de natureza estritamente colunar (data warehousing com agregações sobre bilhões de linhas), domínio em que bancos colunares como Cassandra ou motores OLAP especializados costumam apresentar desempenho superior, assim como para cenários de chave-valor extremamente simples e de altíssima escala, nos quais soluções dedicadas como Redis ou DynamoDB podem ser mais eficientes.

5. Ferramentas de Gerenciamento e Ecossistema

O ecossistema do ArangoDB reúne ferramentas de administração, desenvolvimento e operação:

- Web UI: console administrativo gráfico, com editor AQL interativo, visualizador de grafos (Graph Visualizer) e gerenciamento de coleções, índices e usuários.
- arangosh: shell de linha de comando baseado em JavaScript, usado para administração e automação via scripts.
- arangodump / arangorestore: utilitários de backup e restauração consistente de bancos, inclusive em clusters distribuídos.
- arangoimport / arangoexport: ferramentas de carga e exportação em massa de dados (JSON, CSV, TSV).
- Foxx Microservices: framework que permite escrever endpoints REST em JavaScript executados diretamente dentro do banco (motor V8), reduzindo a necessidade de uma camada de backend separada para operações simples.
- ArangoSearch: motor de busca textual integrado, com ranqueamento de relevância, facetagem e suporte a busca geoespacial e vetorial nativamente em AQL.
- Drivers oficiais: bibliotecas de conexão para JavaScript (arangojs), Python (python-arango, utilizado na aplicação prática deste trabalho), Java, Go, C#, PHP, Ruby e Rust.

- kube-arangodb (Kubernetes Operator): automatiza a implantação e operação de clusters ArangoDB em ambientes Kubernetes.
- Arango Managed Platform (plataforma gerenciada em nuvem, antiga ArangoGraph Insights Platform/Oasis): oferta SaaS com provisionamento automático em provedores de nuvem, reduzindo o esforço operacional.

6. Execução Prática com Docker

A forma mais simples de executar o ArangoDB é por meio de um container Docker oficial. O arquivo docker-compose.yml a seguir (incluído no projeto que acompanha este documento) sobe uma instância completa, expondo a porta padrão 8529:

```
version: "3.8"

services:
  arangodb:
    image: arangodb:3.12
    container_name: arangodb_crud
    ports:
      - "8529:8529"
    environment:
      - ARANGO_ROOT_PASSWORD=openSesame123
    volumes:
      - arangodb_data:/var/lib/arangodb3
      - arangodb_apps:/var/lib/arangodb3-apps

volumes:
  arangodb_data:
  arangodb_apps:
```

Comandos para execução durante a apresentação:

- `docker compose up -d` — inicia o container em segundo plano.
- `docker ps` — confirma que o container está em execução.
- `docker logs arangodb_crud --tail 50` — acompanha a inicialização do servidor.
- Acesso à Web UI em `http://localhost:8529`, com usuário `root` e a senha definida na variável `ARANGO_ROOT_PASSWORD`.
- `docker compose down -v` — finaliza o container e remove os volumes de dados ao término da demonstração.

7. Aplicação CRUD em Python (python-arango)

A aplicação prática que acompanha este trabalho implementa um cenário de loja virtual (clientes e pedidos), cobrindo as quatro operações de CRUD e demonstrando, em AQL, os equivalentes das operações de consulta e agregação solicitadas no roteiro do trabalho: `find`, `$match`, `$project`, `$lookup`, `$unwind`, `$group` e o conceito geral de pipeline de `aggregate`.

7.1 Modelo de Dados

A coleção ‘clientes’ armazena um subdocumento ‘endereco’ (rua, número, cidade, estado, CEP) e um array ‘interesses’. A coleção pedidos armazena um array de subdocumentos ‘itens’ (produto, quantidade, preço unitário) e uma referência `cliente_id` para o cliente que originou o pedido. Uma terceira coleção, de arestas (comprou), liga clientes a pedidos, formando um grafo que demonstra, na prática, a natureza multi-modelo do banco.

7.2 Estrutura do Projeto

Arquivo	Responsabilidade
docker-compose.yml	Sobe o ArangoDB localmente.
config.py / db_connection.py	Configurações e abertura da conexão autenticada.
setup_database.py	Cria coleções, índices e o grafo grafo_vendas.
seed_data.py	Insere dados de exemplo (clientes, pedidos, arestas).
crud_clientes.py / crud_pedidos.py	CRUD completo das duas coleções de documentos.
consultas_avancadas.py	Equivalentes AQL de find/\$match/\$project/\$unwind/\$group/\$lookup.
main.py	Roteiro de demonstração ponta a ponta para a apresentação.

7.3 Operações CRUD

As operações de Create, Read, Update e Delete são implementadas tanto com os métodos nativos do driver python-arango (insert, get, update, delete e find — este último já reproduzindo diretamente a semântica do find() popularizado pelo MongoDB) quanto com AQL puro, para os casos em que é necessário manipular arrays e subdocumentos, como adicionar um item a um pedido existente ou um novo interesse a um cliente sem duplicidade.

7.4 Mapeamento de Conceitos: estilo MongoDB → AQL (ArangoDB)

No ArangoDB não existem comandos separados para "find" e "aggregate": toda consulta — incluindo filtro, projeção, junção, explosão de array e agrupamento — é expressa em uma única linguagem (AQL), combinando as cláusulas FOR, FILTER, LET, COLLECT e RETURN. A tabela a seguir resume a equivalência entre os termos pedidos no roteiro do trabalho e sua expressão em AQL:

Conceito (estilo MongoDB)	Equivalente em AQL (ArangoDB)
find({...})	FOR doc IN colecao FILTER ... RETURN doc
\$match	FILTER
\$project	RETURN { campo: doc.campo, ... }
\$unwind	FOR item IN doc.array
\$group / \$sum / \$count	COLLECT campo = ... AGGREGATE total = SUM(...) ou WITH COUNT INTO
\$lookup (junção manual)	Segundo FOR + FILTER comparando chaves
\$lookup (via relacionamento)	Travessia de grafo: FOR v IN 1..1 OUTBOUND inicio edgeCollection
aggregate([...]) (pipeline)	Uma única consulta AQL combinando FOR / FILTER / COLLECT / LET / RETURN

7.5 Exemplo de Pipeline Completo

O trecho a seguir reproduz, em uma única consulta AQL, um pipeline equivalente a \$match + \$unwind + \$group + \$lookup + \$project: o total gasto por cliente, somente em pedidos concluídos, já trazendo o nome e a cidade do cliente:

```
// $match: somente pedidos concluídos
FOR p IN pedidos
  FILTER p.status == "concluido"

  // $unwind: abre o array de itens
  FOR item IN p.itens

    // $group: soma por cliente
    COLLECT cliente_id = p.cliente_id
    AGGREGATE total_gasto = SUM(item.quantidade * item.preco_unitario)

    // $lookup: busca o documento do cliente
    LET cliente = DOCUMENT(cliente_id)

    // $project: formata o resultado final
    RETURN {
      cliente: cliente.nome,
      cidade: cliente.endereco.cidade,
      total_gasto: total_gasto
    }
```

8. Conclusão

O ArangoDB se diferencia no cenário NoSQL por unificar, em um único motor e em uma única linguagem de consulta, os modelos de documento, grafo e chave-valor. Em clusters, sua arquitetura prioriza consistência sobre disponibilidade em caso de partição de rede (classificação CP), o que o torna especialmente adequado a domínios com relacionamentos complexos e necessidade de integridade dos dados, como redes sociais, detecção de fraude, gestão de identidade e catálogos de e-commerce. A aplicação prática desenvolvida neste trabalho demonstrou, de forma direta, como operações populares de consulta e agregação (find, \$match, \$project, \$unwind, \$group e \$lookup), são todas resolvidas dentro da própria AQL, incluindo uma alternativa nativa baseada em grafo para o \$lookup, evidenciando a vantagem prática do modelo multi-modelo.

9. Referências

ARANGODB. Cluster deployments. Documentação oficial. Disponível em: docs.arangodb.com/3.11/deploy/deployment/cluster/.

ARANGODB. Repositório oficial no GitHub. Disponível em: github.com/arangodb/arangodb.

ARANGO. Documentação oficial da plataforma (AQL, Foxx, Graph Visualizer, Arango Managed Platform). Disponível em: docs.arango.ai.

PYTHON-ARANGO. Documentação do driver oficial em Python. Disponível em: docs.python-arango.com.

GILBERT, S.; LYNCH, N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News, 2002 (formulação original do teorema CAP).